

UTILISER L'IA POUR PRÉVOIR LES RUPTURES D'ANÉVRISME

Présentation de TIPE

Elie Pernet

1^{er} juillet 2026

Problématique

Comment bâtir des modèles d'IA capables de calculer efficacement le comportement sanguin dans un anévrisme intracrânien afin d'en prévoir la rupture?

I. Modélisation du problème

- 1.1. 1. Les anévrismes cérébraux
- 1.2. 2. Les équations de Navier Stokes
- 1.3. 3. Le DataSet AnXplore
- 1.4. 4. Écoulement de Kovasznay

II. Les réseaux de neurones appliqués à la physique

- 2.1. 1. Le perceptron multi-couche
- 2.2. 2. Les Physics Informed Neural Networks
- 2.3. 3. Application à un anévrisme

III. Gagner en généralité temporelle

- 3.1. 1. Le Transfer learning
- 3.2. 2. Les DeepOperatorNetworks

IV. Conclusion

SECTION 1

Modélisation du problème

1. LES ANÉVRISMES CÉRÉBRAUX

Qu'est-ce qu'un anévrisme?

Dilatation sous forme de ballon d'une artère cérébrale

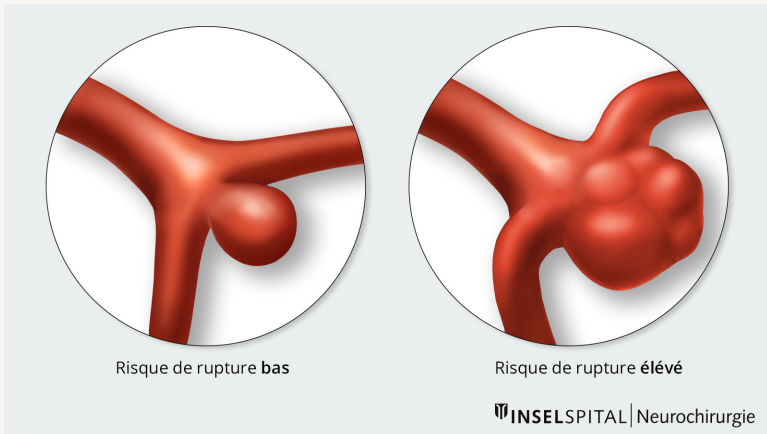


Figure 1 : [I1]

1. LES ANÉVRISMES CÉRÉBRAUX

WSS – Contrainte de cisaillement pariétale

$$\tau_w(t) = \nu \left. \frac{\partial u}{\partial n} \right|_{\text{paroi}}$$

TAWSS – Contrainte de cisaillement moyenne temporelle

$$\frac{1}{T} \int_0^T |\tau_w(t)| dt$$

OSI – Indice de cisaillement oscillatoire

$$\frac{1}{2} \left(1 - \frac{\left| \int_0^T \tau_w dt \right|}{\int_0^T |\tau_w| dt} \right)$$

RRT – Temps de résidence relatif

$$\frac{1}{(1 - 2 \cdot \text{OSI}) \cdot \text{TAWSS}}$$

1. LES ANÉVRISMES CÉRÉBRAUX

I Objectifs

- ◉ Les indicateurs sont faciles à calculer : simple intégration numérique
- ◉ Réaliser un solveur pour calculer (u, v, p) pour :
 - ◊ N'importe quelle géométrie
 - ◊ N'importe quel instant d'un cycle cardiaque
 - ◊ Dans un temps raisonnable (quelques minutes)
- ◉ Solveurs classiques sont *patient specific* : tout doit être recalculé pour chaque nouveau patient.

2. LES ÉQUATIONS DE NAVIER STOKES

■ En stationnaire et en projection 2D

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0$$

$$u \frac{\partial u}{\partial x} + v \frac{\partial u}{\partial y} = -\frac{1}{\rho} \frac{\partial P}{\partial x} + \nu \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right)$$

$$u \frac{\partial v}{\partial x} + v \frac{\partial v}{\partial y} = -\frac{1}{\rho} \frac{\partial P}{\partial y} + \nu \left(\frac{\partial^2 v}{\partial x^2} + \frac{\partial^2 v}{\partial y^2} \right)$$

Viscosité cinématique du fluide : $\nu = \frac{\eta}{\rho}$

2. LES ÉQUATIONS DE NAVIER STOKES

I Variables et équations adimensionnées

$$\mathbf{x}^* = \frac{\mathbf{x}}{L} \quad \mathbf{u}^* = \frac{\mathbf{u}}{U_\infty} \quad p^* = \frac{p}{\rho U_\infty^2}$$

$$\nabla^* \cdot \mathbf{u}^* = 0$$

$$\frac{\partial \mathbf{u}^*}{\partial t^*} + (\mathbf{u}^* \cdot \nabla^*) \mathbf{u}^* = -\nabla^* p^* + \frac{1}{Re} \nabla^{*2} \mathbf{u}^* \quad Re = \frac{U_\infty L}{\nu}$$

3. LE DATASET ANXPLORE

- ◉ Dataset de 100 modèles 3D normalisés d'anévrismes cérébraux.
- ◉ Projet de recherche du CEMEF (Centre de mise en forme des matériaux - CNRS UMR 7635)
- ◉ Fournit des résolutions complètes dégradées

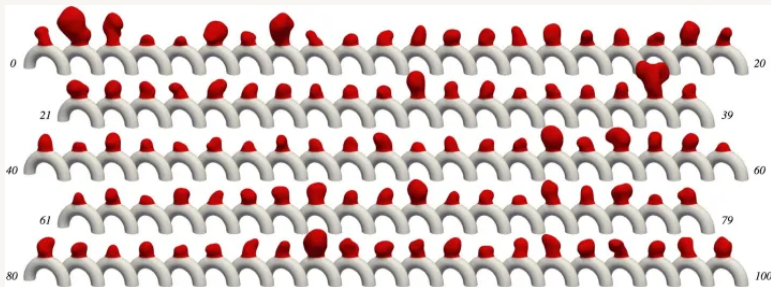


Figure 2 : [12]

4. ÉCOULEMENT DE KOVASZNAY

■ Un cas de résolution exacte de Navier Stokes

Modélise l'écoulement devant une grille 2D

$$u(x, y) = 1 - e^{\lambda x} \cos(2\pi y) \quad v(x, y) = \frac{\lambda}{2\pi} e^{\lambda x} \sin(2\pi y)$$

$$P(x, y) = \frac{1}{2} \left(1 - e^{2\lambda x} \right)$$

■ Paramètre λ

$$\lambda = \frac{Re}{2} - \sqrt{\frac{Re^2}{4} + 4\pi^2}$$

SECTION 2

Les réseaux de neurones appliqués à la physique

1. LE PERCEPTRON MULTI-COUCHE

Le perceptron de Rosenblatt (1957)

Prédire après une boucle d'entraînement sur des données labellisées une sortie correcte sur des données inconnues.

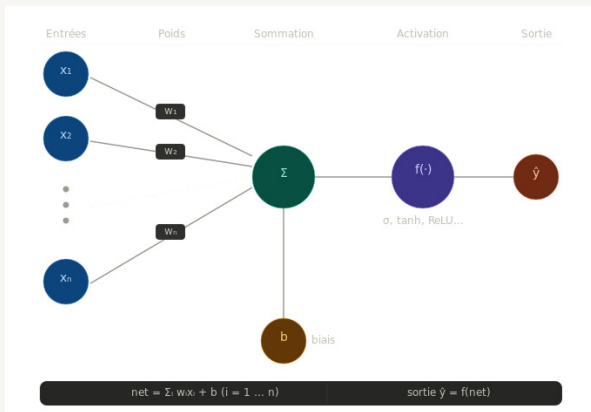


Figure 3

1. LE PERCEPTRON MULTI-COUCHE

I L'algorithme d'entraînement

- ❶ Le modèle initialise via un algorithme les poids initiaux
- ❷ On répète n fois la même étape :
 - ◊ On évalue la sortie du modèle avec nos données labellisées
 - ◊ Calcul de l'erreur entre la sortie actuelle et les données labellisées (méthode des moindres carrés)
 - ◊ L'optimisateur (Adam ou L-BFGS ici) adapte les poids
- ❸ Le modèle est entraîné, il est prêt à prédire la sortie pour des données inconnues

1. LE PERCEPTRON MULTI-COUCHE

Le perceptron multicouche

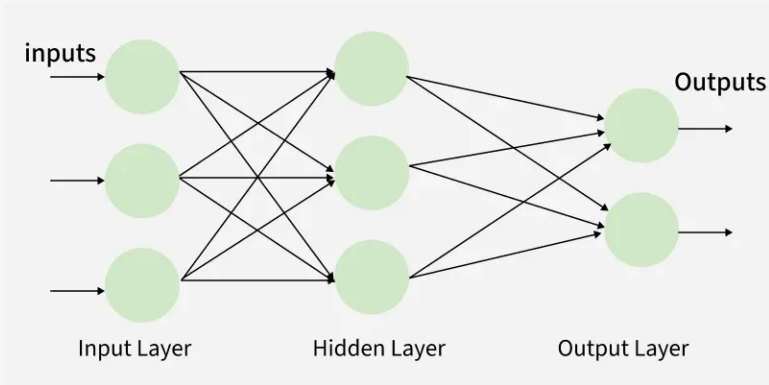


Figure 4

1. LE PERCEPTRON MULTI-COUCHE

I Théorème d'approximation universelle

Pour un réseau à une couche cachée :

$$g(\mathbf{x}) = \sum_{j=1}^m w_j^{(2)} \sigma \left(\sum_{i=1}^n x_i w_{j,i}^{(1)} - b_j \right) = \sum_{j=1}^m w_j^{(2)} \sigma \left(\mathbf{x}^\top \mathbf{w}_j^{(1)} - b_j \right)$$

- ⊙ $\mathbf{x} \in \mathbb{R}^n$ – vecteur d'entrée
- ⊙ σ – fonction d'activation non-linéaire
- ⊙ $w_{j,i}^{(1)} \in \mathbb{R}$ – poids du neurone i vers j (couche cachée), entrées de $\mathbf{w}_j^{(1)} \in \mathbb{R}^n$
- ⊙ $b_j \in \mathbb{R}$ – biais du neurone j
- ⊙ $w_j^{(2)} \in \mathbb{R}$ – poids de sortie du neurone j
- ⊙ m – nombre de neurones dans la couche cachée

Pour tout $f \in \mathcal{C}(K)$ avec $K \subset \mathbb{R}^n$ compact et tout $\varepsilon > 0$, il existe un réseau de neurones à une couche cachée $\hat{f}$ tel que :

$$\sup_{\mathbf{x} \in K} |f(\mathbf{x}) - g(\mathbf{x})| < \varepsilon$$

1. LE PERCEPTRON MULTI-COUCHE

I Techniquement

- ◉ Bibliothèques **PyVista** et **H5py** pour lire les fichiers 3D et les résultats de simulation
- ◉ Bibliothèque **PyTorch** pour créer et entraîner les réseaux de neurones.
- ◉ Bibliothèque **Matplotlib** pour afficher les résultats
- ◉ **Kaggle** pour entraîner les modèles (cpu ou gpu)

2. LES PHYSICS INFORMED NEURAL NETWORKS

I La fonction de perte

- ⊙ Des conditions aux limites :
 - ◇ Sur la solution (conditions de Dirichlet)
 - ◇ Sur les dérivées spatiales et temporelle de la solution (conditions de Neumann)
- ⊙ Les résidus d'équations différentielles aux dérivées partielles
- ⊙ De potentielles données expérimentales réparties sur le domaine de résolution

2. LES PHYSICS INFORMED NEURAL NETWORKS

I Objectif

Apprendre la solution exacte sur un domaine 2D :

$$\Omega = [-0.5, 1] \times [-0.5, 1.5]$$

I Architecture du modèle

- ❶ 2 entrées : (x, y)
- ❷ 2 couches cachées de 64 neurones
- ❸ 3 sorties : (u, v, p)
- ❹ Activation avec $\tanh$ entre chaque couche

I L'entraînement du modèle

- ⦿ 1000 étapes avec Adam : faire converger la solution
- ⦿ 1000 étapes avec L-BFGS : affiner la solution

2. LES PHYSICS INFORMED NEURAL NETWORKS

Points d'entraînement

- ◉ 100 points répartis linéairement sur le contour où le modèle connaît la solution exacte.
- ◉ 2000 points aléatoires dans Ω où Navier Stokes sera évalué.

Fonction de perte

$$\mathcal{L}_{tot} = \mathcal{L}_{outside} + \mathcal{L}_{continuity} + \mathcal{L}_{residuals}$$

$$\mathcal{L}_{tot} = \frac{\lambda_1}{N_{bc}} \sum_{i=1}^{N_{bc}} (m(x_i, y_i) - m_{bc}(x_i, y_i))^2$$
$$+ \frac{\lambda_2}{N_{ns}} \sum_{i=1}^{N_{ns}} (continuity(x_i, y_i)^2 + residuals(x_i, y_i)^2)$$

$$\lambda_1 = 100 \quad \lambda_2 = 1$$

2. LES PHYSICS INFORMED NEURAL NETWORKS

■ Résultats pour $Re = 60.7$ minutes 59 secondes sur CPU

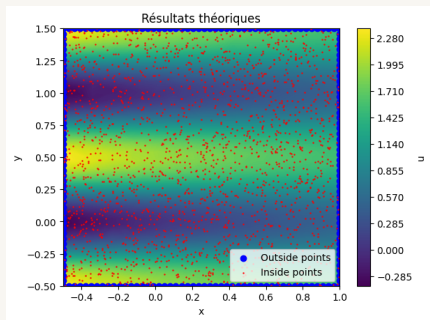


Figure 5

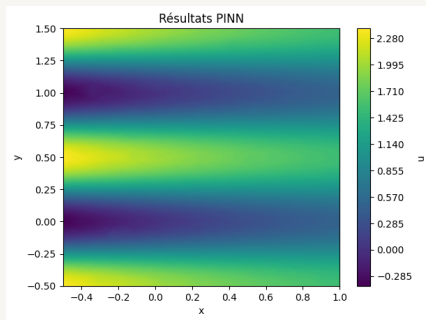


Figure 6

2. LES PHYSICS INFORMED NEURAL NETWORKS

Évolution de l'erreur

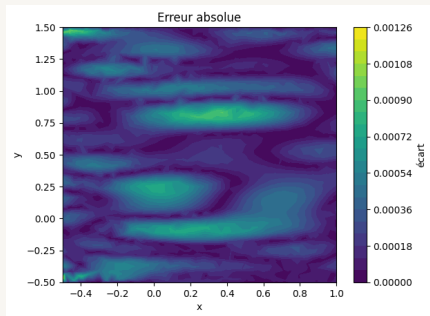


Figure 7

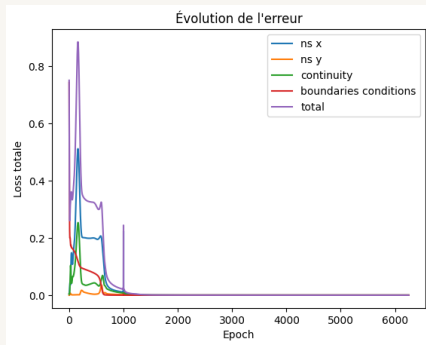


Figure 8

2. LES PHYSICS INFORMED NEURAL NETWORKS

■ Comparaison de *Adam* et *L – BFGS*

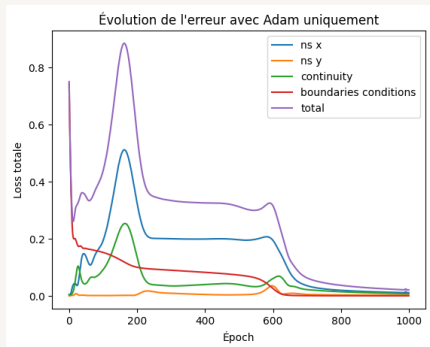


Figure 9

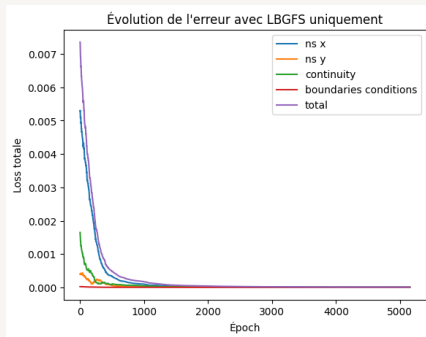


Figure 10

3. APPLICATION À UN ANÉVRISME

Conditions aux limites

- Non-glissement :

$$\mathbf{v}(M) = \mathbf{0} \quad \forall M \in \partial\Omega_{\text{wall}}$$

- Profil parabolique en entrée :

$$v_y(M) = \frac{V_{\text{max}}}{V_{\text{norm}}} \left(1 - \frac{r(M)}{R} \right)$$

$$v_x = 0 \quad \forall M \in \partial\Omega_{\text{inlet}}$$

- Pression de sortie (résistance hydraulique) :

$$P(M) = P_0 + R_d Q(t) \quad \forall M \in \partial\Omega_{\text{outlet}}$$

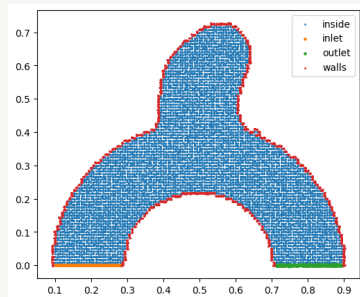
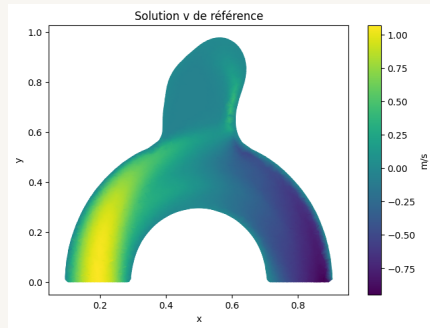
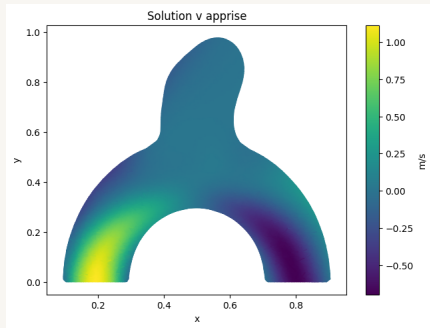


Figure 11 : Discretisation du domaine Ω

3. APPLICATION À UN ANÉVRISME

■ Résultats



3. APPLICATION À UN ANÉVRISME

■ Erreur

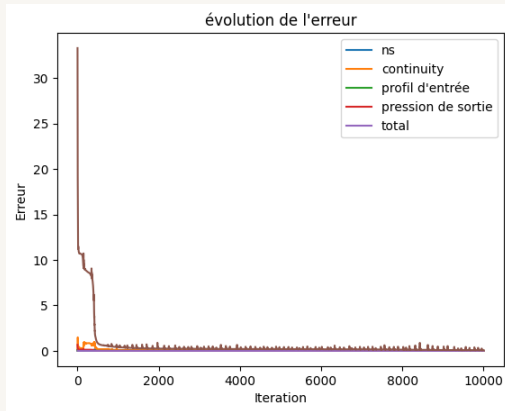


Figure 14

3. APPLICATION À UN ANÉVRISME

Modification du modèle

- ⊙ Apprendre la pression en fournissant partout les vitesses dans les deux directions.
- ⊙ Pic systolique

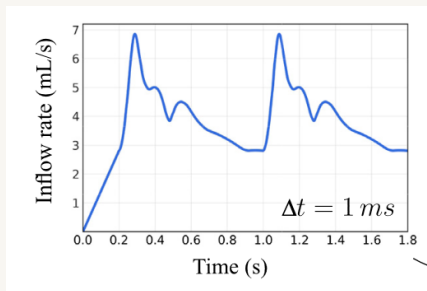


Figure 15 : [14]

3. APPLICATION À UN ANÉVRISME

■ Résultats pour v et p

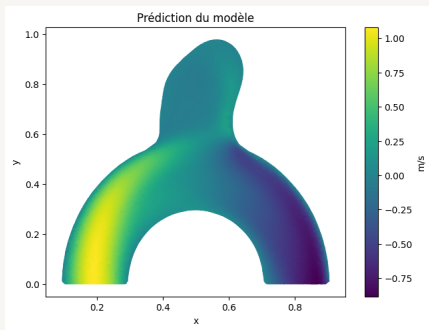


Figure 16

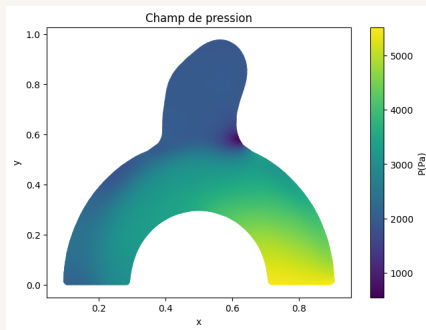


Figure 17

3. APPLICATION À UN ANÉVRISME

Observation de l'erreur

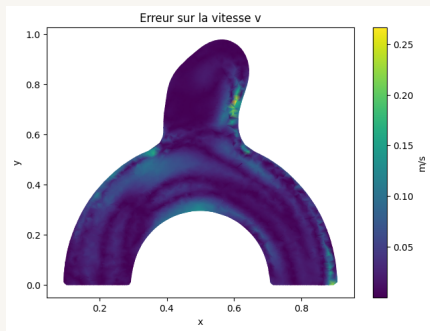


Figure 18

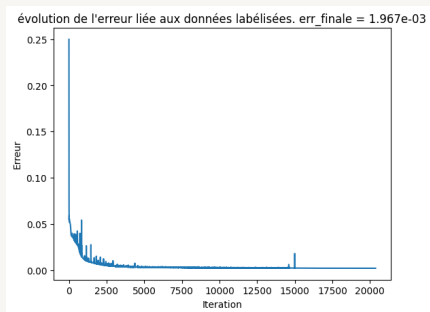


Figure 19

SECTION 3

Gagner en généralité temporelle

1. LE TRANSFER LEARNING

I Le *fine-tuning* de modèle

- ◉ Modèle entraîné dans une géométrie donnée et à un instant donné.
- ◉ Ne répond pas au critère de rapidité
- ◉ Partir de la solution existante et la faire converger vers de nouvelles conditions aux limites
- ◉ Principe utilisé pour entraîner les LLM actuels

1. LE TRANSFER LEARNING

I Pour l'anévrisme

- ⊙ On reprend le modèle de la partie I.3 déjà entraîné
- ⊙ Nouvelle phase d'entraînement sur 2000 itérations Adam et 1000 L-BFGS
- ⊙ Nouveau profil d'entrée et nouvelle pression de sortie

1. LE TRANSFER LEARNING

Premier essai : solution triviale apprise immédiatement

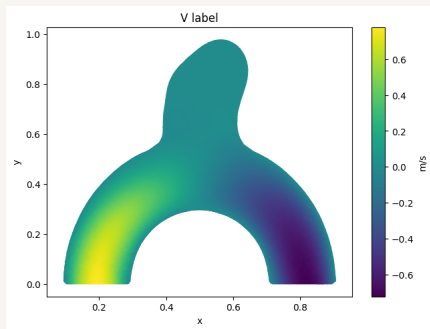


Figure 20

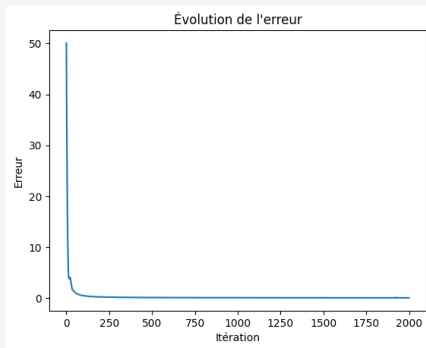


Figure 21

1. LE TRANSFER LEARNING

I Contrôler l'apprentissage

- ⊙ Pas d'apprentissage *learning rate* 1000 fois plus petit
- ⊙ Nouveau terme d'erreur : de combien le réseau s'est éloigné des poids initiaux
- ⊙ On le pondère via une exponentielle décroissante en le nombre d'itérations
 - ◇ **0 -> 1600 Pas de physique :**
 $\lambda_{phy} = 0$ et $\lambda_{anchor} = 1e5 \rightarrow 2e4$
 - ◇ **1600 -> 4800 Réintroduction progressive de la physique :**
 $\lambda_{phy} = 0 \rightarrow 1$ et $\lambda_{anchor} = 2e4 \rightarrow 5e2$
 - ◇ **4800 -> 8000 On libère les poids initiaux :**
 $\lambda_{phy} = 1$ et $\lambda_{anchor} = 5e2 \rightarrow 1e2$
 - ◇ **Phase L-BFGS :**
 $\lambda_{phy} = 1$ et $\lambda_{anchor} = 0$
- ⊙ Poids figés
- ⊙ Physique réintroduite progressivement

1. LE TRANSFER LEARNING

■ Comparaison de l'évolution de l'erreur

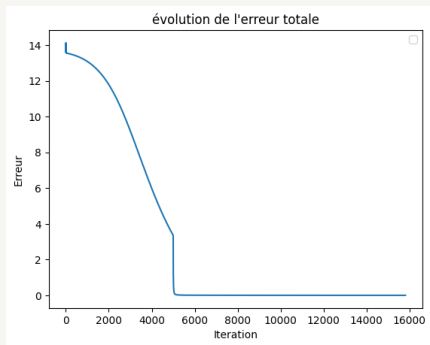


Figure 22

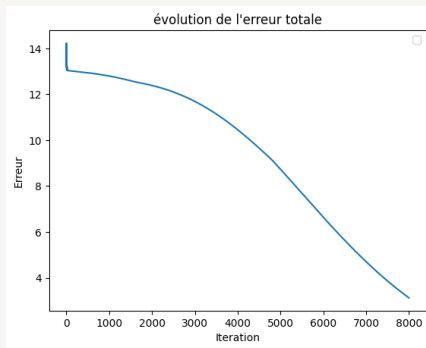


Figure 23

1. LE TRANSFER LEARNING

Résultat. 4h20 sur GPU

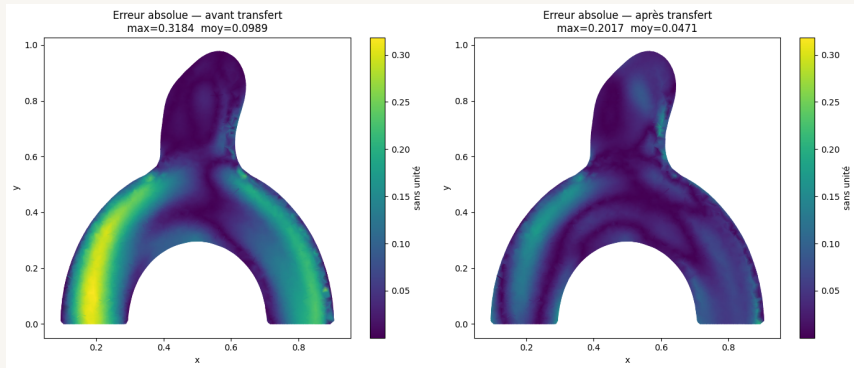


Figure 24

2. LES DEEPOPATORNETWORKS

I Objectifs

- ◉ Ne pas entraîner de nouveaux modèle en permanence
- ◉ Apprendre au modèle comment tel ou tel paramètre influence la solution
- ◉ Entraîner un seul gros modèle

I Principes

- ◉ Apprendre des opérateurs et non plus des fonctions
- ◉ On donne $Q(t)$ en entrée et on obtient en sortie u, v et p en (x, y, t)
- ◉ Exemple : apprendre l'opérateur d'intégration ou la solution d'une équation de diffusion avec un terme de source

2. LES DEEPOPERATORNETWORKS

I Principes

- ⊙ Apprendre l'influence du nombre de Reynolds sur la solution
- ⊙ Fournir la solution pour tout Reynolds en une seule phase d'entraînement
- ⊙ Modèle sur-dimensionné, la fonction d'entrée est une fonction constante

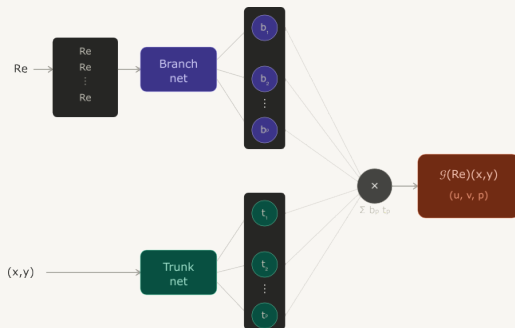


Figure 25

2. LES DEEPOPATORNETWORKS

Résultats. 3h07 sur GPU

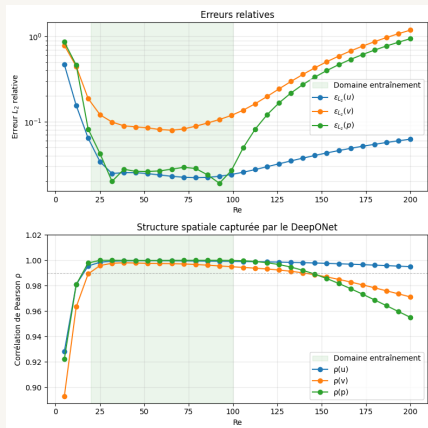


Figure 26

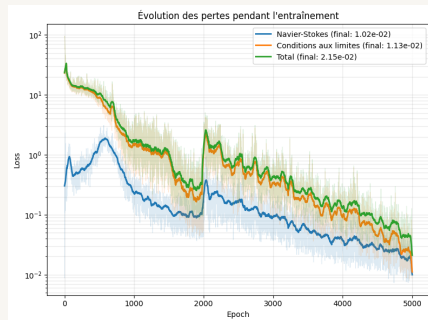


Figure 27

2. LES DEEPOPATORNETWORKS

I Limites

- ⊙ Nécessite un grand nombre de données d'entraînement : simulation complète pour un grand nombre volumique de $Q(t)$
- ⊙ Temps d'entraînement très long
- ⊙ Très grande capacité de calcul nécessaire

I Questionnement du modèle

- ⊙ Long temps d'entraînement, ne répond pas au besoin d'avoir des résultats rapidement
- ⊙ N'encode que le cycle cardiaque et non la géométrie
- ⊙ Généralisation uniquement temporelle
- ⊙ Pas de caractérisation de chaque géométrie par un paramètre simple : pas d'encodage via DeepONet possible

SECTION 4

Conclusion

I Démarche

- ⊙ **Premier modèle** : orienté physique
 - ◇ Converge vers une solution triviale : pas assez d'information
- ⊙ **Deuxième modèle** : orienté données
 - ◇ Apprend la solution mais nécessite des données : point de départ pour du *transfer learning*
- ⊙ **Troisième modèle** : *Transfer Learning*
 - ◇ Permet d'apprendre la solution à n'importe quel t mais grand temps d'entraînement
- ⊙ **Quatrième modèle** : *DeepOperatorNetwork*
 - ◇ Grand temps d'entraînement mais permet ensuite d'avoir la solution pour n'importe quel cycle cardiaque

I Analyse globale des résultats

- ⊙ Modèles efficaces et pertinents dans un cas simple
- ⊙ Résultats uniquement 2D : indicateurs non représentatifs.
- ⊙ Aucune prise en compte du flux tourbillonnant et de la courbure hors plan.
- ⊙ La paroi artérielle a été prise fixe : elle est en réalité en mouvement.
- ⊙ Calcul de la contrainte de Cauchy à rupture : mécanique des matériaux.

CONCLUSION

Perspectives

- ◉ Utiliser les MeshGraphNets : réseaux basés sur les graphes (maillage)
- ◉ Simuler le comportement de l'anévrisme après la mise en place de diverses solutions

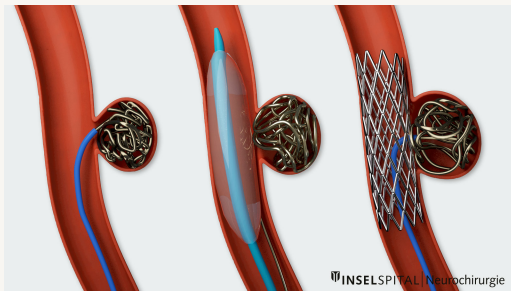


Figure 28 : [15]

Annexes

A1 — SOURCES DES ILLUSTRATIONS

[I1, I5] neurochirurgie.insel.ch

[I2, I4] frontiersin.org

[I3] geeksforgeeks.org

Les hypothèses

Écoulement incompressible et homogène d'un fluide de viscosité constante η et de masse volumique constante ρ

Conservation de la masse

$$\frac{\partial \rho}{\partial t} + \operatorname{div}(\rho \vec{v}) = 0 \Leftrightarrow \operatorname{div}(\vec{v}) = 0 \quad (1)$$

A2 - DÉMONSTRATION NAVIER-STOKES

I Conservation de la quantité de mouvement

Système : particule fluide de volume $d\tau$, de masse volumique ρ

Accélération particulaire :

$$\frac{D\vec{v}}{Dt} = \frac{\partial\vec{v}}{\partial t} + (\vec{v} \cdot \nabla)\vec{v}$$

Bilan des forces :

- ◉ Pression : $-\nabla P$
- ◉ Viscosité : $\eta \Delta\vec{v}$
- ◉ Pesanteur négligée devant les autres forces

$$\rho \frac{D\vec{v}}{Dt} = -\nabla P + \eta \Delta\vec{v} \quad (2)$$

I Application à l'écoulement de Kovasznay

- ◉ On connaît la solution complète pour $Re = 40$, on souhaite apprendre la solution pour $Re = 60$
- ◉ Le modèle est le même qu'en partie I
- ◉ Rajout d'une boucle d'entraînement une fois la première terminée
- ◉ On utilise uniquement LBFGS sur 1000 epochs

A3 - TRANSFER LEARNING POUR L'ÉCOULEMENT DE KOVASZNY

Résultats pour $Re = 60$. 3 minutes 12 secondes sur CPU

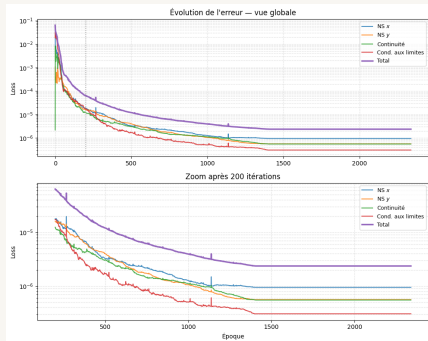


Figure 29

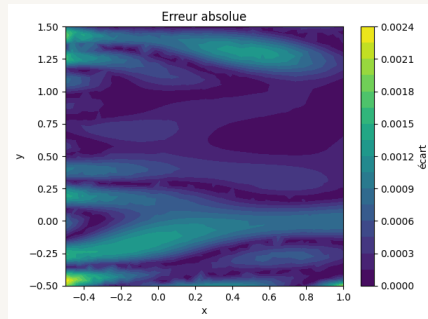


Figure 30

A4 - ANALYSE DES INDICATEURS

I TAWSS – Contrainte de cisaillement moyenne temporelle

Force tangentielle du sang sur la paroi sur un cycle cardiaque.

Trop faible ($< 0,4$ Pa) ou trop élevé (> 3 Pa) $\rightarrow$ augmente le risque de rupture.

I OSI – Indice de cisaillement oscillatoire

Degré de changement de direction du flux. $OSI \in [0; 0,5]$.

$OSI > 0,3 \rightarrow$ augmente le risque de rupture.

I RRT – Temps de résidence relatif

Stagnation du sang près de la paroi (combine TAWSS faible et OSI élevé).

RRT élevé $\rightarrow$ augmente le risque de rupture.

I Rôle de la pression

Pression sanguine : 80–120 mmHg. Pression intracrânienne > 30 mmHg $\rightarrow$ dangereuse.

Une pression élevée dans le dôme aggrave les zones à faible TAWSS et OSI élevé.

I Adam – 1^{er} ordre

- ⊙ Mise à jour des paramètres :

$$\theta \leftarrow \theta - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}$$

α taux d'apprentissage

$\hat{v}_t$ variance récente du gradient

$\hat{m}_t$ direction moyenne du gradient

ε stabilisation numérique

- ⊙ Robuste, explore efficacement, mais converge lentement près du minimum

I L-BFGS – (Broyden, Fletcher, Goldfarb et Shanno)

- ⊙ Quasi Newtonien : approxime le Hessien inverse pour exploiter la courbure locale
- ⊙ Convergence précise mais nécessite une bonne initialisation

```
import json

import h5py
import torch
import pyvista as pv
import numpy as np
from matplotlib import pyplot as plt, tri
import matplotlib.colors as mcolors

def kova_solution_np(x, y, nu):
    l = 1 / (2 * nu) - np.sqrt(1 / (4 * nu**2) + 4 * np.pi**2)

    u = 1 - np.exp(1 * x) * np.cos(2 * np.pi * y)
    v = (1 / (2 * np.pi)) * np.exp(1 * x) * np.sin(2 * np.pi * y)
    p = 0.5 * (1 - np.exp(2 * l * x))

    return np.stack([u, v, p], axis=1)

def kova_solution(x, y, nu) -> torch.Tensor:
    result_np = kova_solution_np(x.detach().cpu().numpy(), y.detach().cpu().numpy(), nu)

    return torch.tensor(result_np, dtype=x.dtype, device=x.device)

def load_losses(filename):
    losses = []

    with open(filename, "r") as f:
```

```
    losses = [i for i in json.load(f) if np.shape(i)]

    return np.array(losses)

def ns_residuals(coords, u, v, p, nu):
    grad_u = torch.autograd.grad(u, coords, torch.ones_like(u), create_graph=True)[0]
    du_x = grad_u[:, 0]
    du_y = grad_u[:, 1]

    grad_v = torch.autograd.grad(v, coords, torch.ones_like(v), create_graph=True)[0]
    dv_x = grad_v[:, 0]
    dv_y = grad_v[:, 1]

    grad_p = torch.autograd.grad(p, coords, torch.ones_like(p), create_graph=True)[0]
    dp_x = grad_p[:, 0]
    dp_y = grad_p[:, 1]

    du_xx = torch.autograd.grad(du_x, coords, torch.ones_like(du_x), create_graph=True)[0][:, 0]
    du_yy = torch.autograd.grad(du_y, coords, torch.ones_like(du_y), create_graph=True)[0][:, 1]

    dv_xx = torch.autograd.grad(dv_x, coords, torch.ones_like(dv_x), create_graph=True)[0][:, 0]
    dv_yy = torch.autograd.grad(dv_y, coords, torch.ones_like(dv_y), create_graph=True)[0][:, 1]

    ns_x = u * du_x + v * du_y + dp_x - nu * (du_xx + du_yy)
    ns_y = u * dv_x + v * dv_y + dp_y - nu * (dv_xx + dv_yy)
    continuity = du_x + dv_y

    return ns_x, ns_y, continuity
```

```
def extract_ds_values(file_name, T):  
    with h5py.File(file_name, "r") as f:  
        coords = f["data0"][:]  
        connectivity = f["data1"][:]  
        vitesses = f[f"data{2 + 2 * T}"][:]  
  
    cells = np.hstack([np.full((len(connectivity), 1), 4), connectivity]).ravel()  
    cell_types = np.full(len(connectivity), pv.CellType.TETRA)  
    mesh = pv.UnstructuredGrid(cells, cell_types, coords)  
    mesh.point_data["Vitesse"] = vitesses  
  
    slice_plane = mesh.slice(normal='z')  
  
    points = slice_plane.points[:, :2]  
    U = slice_plane.point_data["Vitesse"]  
    u = U[:, 0]  
    v = U[:, 1]  
  
    uv_list = np.stack([u, v], axis=1)  
    x = points[:, 0]  
    y = points[:, 1]  
  
    x = np.array(list(map(lambda x_c: x_c + 10, x)))  
    return x, y, uv_list  
  
def extract_ds_outside_values(file_name):  
    with h5py.File(file_name, "r") as f:  
        coords = f["data0"][:]
```

```

connectivity = f["data1"][:]

cells = np.hstack([np.full((len(connectivity), 1), 4), connectivity]).ravel()
cell_types = np.full(len(connectivity), pv.CellType.TETRA)
mesh = pv.UnstructuredGrid(cells, cell_types, coords)

slice_plane = mesh.slice(normal='z')
edges = slice_plane.extract_feature_edges(boundary_edges=True,
                                          non_manifold_edges=False,
                                          feature_edges=False,
                                          manifold_edges=False)

contour_points = edges.points
x = np.array(list(map(lambda x_c: x_c + 10, contour_points[:, 0])))
return x, contour_points[:, 1]

def plot_field(x, y, z, plot_title, z_label, show_norm=True):
    triang = tri.Triangulation(x, y)
    triangles = triang.triangles

    x_tri = x[triangles]
    y_tri = y[triangles]

    a = np.sqrt((x_tri[:, 1] - x_tri[:, 0]) ** 2 + (y_tri[:, 1] - y_tri[:, 0]) ** 2)
    b = np.sqrt((x_tri[:, 2] - x_tri[:, 1]) ** 2 + (y_tri[:, 2] - y_tri[:, 1]) ** 2)
    c = np.sqrt((x_tri[:, 0] - x_tri[:, 2]) ** 2 + (y_tri[:, 0] - y_tri[:, 2]) ** 2)

    max_edge = np.max(np.stack([a, b, c], axis=1), axis=1)

    threshold = 0.05

```



```
triang.set_mask(max_edge > threshold)

# --- Plot ---
fig, ax = plt.subplots()
if show_norm:
    norm = mcolors.TwoSlopeNorm(vmin=z.min(), vcenter=0, vmax=z.max())
    tcf = ax.tripcolor(triang, z, cmap='RdBu_r', shading='gouraud', norm=norm)
else:
    tcf = ax.tripcolor(triang, z, cmap='viridis', shading='gouraud')
ax.set_aspect('auto')
ax.set_xlim(x.min() - 0.05, x.max() + 0.05)
ax.set_ylim(y.min() - 0.05, y.max() + 0.05)
plt.colorbar(tcf, ax=ax, label=z_label)
plt.title(plot_title)
plt.tight_layout()
plt.xlabel("x")
plt.ylabel("y")
plt.show()

def kova_data():
    outside_points = torch.Tensor(
        [[-0.5, i] for i in np.linspace(-0.5, 1.5, 25)] +
        [[1, i] for i in np.linspace(-0.5, 1.5, 25)] +
        [[i, -0.5] for i in np.linspace(-0.5, 1, 25)] +
        [[i, 1.5] for i in np.linspace(-0.5, 1, 25)]).double()
    inside_points = torch.tensor(
        np.column_stack((np.random.uniform(-0.5, 1.0, 2000), np.random.uniform(-0.5, 1.5, 2000))))
    ).double().requires_grad_(True)

    return outside_points, inside_points
```

```
def u_func(x, y):  
    return 1 - torch.exp(1 * x) * torch.cos(2 * torch.pi * y)  
  
def v_func(x, y):  
    return (1 / (2 * torch.pi)) * torch.exp(1 * x) * torch.sin(2 * np.pi * y)  
  
def p_func(x):  
    return 1 / 2 * (1 - torch.exp(2 * 1 * x))  
  
def parabolic_inlet(x, V_max):  
    x_center = (x.min() + x.max()) / 2  
    R = (x.max() - x.min()) / 2  
  
    return V_max * (1 - ((x - x_center) / R) ** 2)  
  
def increase_points_number(points, n_variations=8, step_size=0.05, noise_std=0.02):  
    augmented = [points]  
  
    for _ in range(n_variations):  
        # Pas aléatoire dans [-step_size, step_size]  
        random_step = (torch.rand_like(points) * 2 - 1) * step_size  
        # Ajouter un peu de bruit  
        noise = torch.randn_like(points) * noise_std  
        # Nouveaux points  
        new_points = points + random_step + noise  
        augmented.append(new_points)  
  
    return torch.cat(augmented, dim=0)
```

```
import torch.nn as nn
import torch

class KovaDeepONet(nn.Module):
    def __init__(self, p=128, re_min=20, re_max=100):
        super(KovaDeepONet, self).__init__()
        self.p = p
        self.re_min = re_min
        self.re_max = re_max

        self.trunk_net = MLP(4, 128, 2, p, final_tanh=False)
        self.branch_net = MLP(4, 128, 1, p * 3)

        self.bias = nn.Parameter(torch.zeros(3).double())
        self._init_weights()

    def _init_weights(self):
        for m in self.modules():
            if isinstance(m, nn.Linear):
                nn.init.xavier_normal_(m.weight)
                nn.init.zeros_(m.bias)

    def forward(self, re, points):
        re_norm = 2 * (re - self.re_min) / (self.re_max - self.re_min) - 1

        b = self.branch_net(re_norm.unsqueeze(-1)).reshape(-1, 3, self.p)
        t = self.trunk_net(points)

        out = torch.einsum('bcp,np->bcn', b, t)
```

```
        return out.permute(0, 2, 1) + self.bias

class MLP(nn.Module):
    def __init__(self, hidden_n, layer_depth, input_n, output_n, final_tanh=False):
        super(MLP, self).__init__()
        self.layers = nn.ModuleList()
        self.final_tanh = final_tanh

        self.layers.append(nn.Linear(input_n, layer_depth))

        for i in range(hidden_n):
            self.layers.append(nn.Linear(layer_depth, layer_depth))

        self.layers.append(nn.Linear(layer_depth, output_n))

    def forward(self, x):
        for i in range(len(self.layers) - 1):
            x = torch.tanh(self.layers[i](x))

        x = self.layers[-1](x)
        if self.final_tanh:
            x = torch.tanh(x) # Tanh final optionnel
        return x
```

```
import torch
import torch.nn as nn
import numpy as np

import json

from src.utils import ns_residuals, kova_data, u_func, v_func, p_func

Re = 40
nu = 1 / Re
l = 1 / (2 * nu) - np.sqrt(1 / (4 * nu ** 2) + 4 * np.pi ** 2)

PINNModel = nn.Sequential(
    nn.Linear(2, 64),
    nn.Tanh(),
    nn.Linear(64, 64),
    nn.Tanh(),
    nn.Linear(64, 64),
    nn.Tanh(),
    nn.Linear(64, 3)
).double()

def compute_total_loss():
    inside_values = PINNModel(inside_points)
    ns_x, ns_y, continuity = ns_residuals(inside_points, inside_values[:, 0], inside_values[:, 1],
    ↪ inside_values[:, 2], 1/Re)

    ns_loss_x_inside, ns_loss_y_inside = mse(ns_x, torch.zeros_like(ns_x)), mse(ns_y,
    ↪ torch.zeros_like(ns_y))
    continuity_loss_inside = mse(continuity, torch.zeros_like(continuity))
```

```
outside_values = PINNModel(outside_points)
outside_loss = mse(outside_values, outside_true_results)

losses.append([ns_loss_x_inside.item(), ns_loss_y_inside.item(), continuity_loss_inside.item(),
↳ outside_loss.item(), ns_loss_x_inside.item() + ns_loss_y_inside.item() +
↳ continuity_loss_inside.item() + outside_loss.item()])

return ns_loss_x_inside + ns_loss_y_inside + continuity_loss_inside + 100 * outside_loss

adam_optimizer = torch.optim.Adam(PINNModel.parameters(), lr=0.001)
optimizer_lbfgs = torch.optim.LBFGS(
    PINNModel.parameters(),
    lr=1.0,
    max_iter=20,
    history_size=50,
    tolerance_grad=1e-7,
    tolerance_change=1e-9,
    line_search_fn="strong_wolfe"
)

mse = nn.MSELoss()

outside_points, inside_points = kova_data()

outside_true_results = torch.stack([
    u_func(outside_points[:, 0], outside_points[:, 1]),
    v_func(outside_points[:, 0], outside_points[:, 1]),
    p_func(outside_points[:, 0])
], dim=1)

adam_epochs = 1000
lbfgs_epochs = 1000
```

```
losses = []

for _ in range(adam_epochs):
    adam_optimizer.zero_grad()
    loss = compute_total_loss()
    loss.backward()
    losses.append(loss.item())
    adam_optimizer.step()

for _ in range(lbgfs_epochs):
    def closure():
        optimizer_lbfgs.zero_grad()
        loss = compute_total_loss()
        loss.backward()
        return loss

    loss = optimizer_lbfgs.step(closure)

torch.save(PINNModel.state_dict(), "./PINN_Model_09.pt")

with open("losses_09.json", "w") as f:
    json.dump(losses, f)
```

```
import torch
import torch.nn as nn
import numpy as np

import json

from models import KovaDeepONet
from utils import kova_solution, ns_residuals

def loss_function(re_values):
    ns_total_loss = torch.tensor(0.0).double()
    bc_total_loss = torch.tensor(0.0).double()

    for re in re_values:
        re_t = torch.tensor([re]).double()
        nu = 1.0 / re

        inside_points = torch.tensor(
            np.column_stack((np.random.uniform(-0.5, 1.0, 2000), np.random.uniform(-0.5, 1.5,
                ↪ 2000)))
        ).double().requires_grad_(True)

        inside_results = model(re_t, inside_points).squeeze(0)

        ns_x, ns_y, continuity = ns_residuals(inside_points, inside_results[:, 0], inside_results[:,
            ↪ 1], inside_results[:, 2], nu)
        ns_total_loss += (mse(ns_x, torch.zeros_like(ns_x)) + mse(ns_y, torch.zeros_like(ns_y)) +
            ↪ mse(continuity, torch.zeros_like(continuity)))

        outside_results = model(re, outside_points).squeeze(0)
        labels = kova_solution(outside_points[:, 0], outside_points[:, 1], nu)
        bc_total_loss += mse(outside_results, labels)
```



```

    n = len(re_values)
    return LAMBDA_NS * ns_total_loss / n, LAMBDA_BC * bc_total_loss / n

mse = nn.MSELoss()

P          = 128      # Dimension de l'espace latent branch/trunk
N_COLLOC   = 2000     # Points intérieurs par Re
N_BC       = 25       # Points par côté frontière
RE_RANGE   = (20, 100)

LAMBDA_NS = 1.0
LAMBDA_BC = 50.0

RE_BATCH_SIZE = 8

outside_points = torch.Tensor([[[-0.5, i] for i in np.linspace(-0.5, 1.5, 25)] + [[1, i] for i in
↪ np.linspace(-0.5, 1.5, 25)] + [[i, -0.5] for i in np.linspace(-0.5, 1, 25)] + [[i, 1.5] for i in
↪ np.linspace(-0.5, 1, 25)]]).double()

model = KovaDeepONet().double()

losses = []

optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=2000)

for epoch in range(5000):
    # Échantillonnage aléatoire de Re à chaque époque
    re_batch = np.random.uniform(*RE_RANGE, RE_BATCH_SIZE).tolist()

    optimizer.zero_grad()

```

```
loss_ns, loss_bc = loss_function(re_batch)
loss = loss_ns + loss_bc
losses.append([loss_ns.item(), loss_bc.item(), loss.item()])
loss.backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
optimizer.step()
scheduler.step()

if epoch % 500 == 0:
    print(f"[{epoch:5d}] NS={loss_ns:.2e} | BC={loss_bc:.2e}")

torch.save(model.state_dict(), "./DON_kova.pt")

with open("losses_09.json", "w") as f:
    json.dump(losses, f)
```

```
import sys

import matplotlib.pyplot as plt
import numpy as np
import pyvista as pv

sys.setrecursionlimit(10000)

class AneurysmData:
    def __init__(self, file, w, h):
        mesh = pv.read(file)
        slice_plane = mesh.slice(normal='z')

        edges = slice_plane.extract_feature_edges(boundary_edges=True)
        contour_points = np.array(edges.points)
        contour_points = np.array([[cell[1], cell[0] + 10] for cell in contour_points])

        self.file = file
        self.w = w
        self.h = h
        self.s_h = 14 / self.h
        self.s_w = 20 / self.w

        self.grid = np.array([[0 for _ in range(self.w)] for _ in range(self.h)])

        for point in contour_points:
            nearest_cell_x = int(np.floor(point[1] / self.s_w))
            nearest_cell_y = int(np.floor(point[0] / self.s_h))
            self.grid[nearest_cell_y][nearest_cell_x] = 1
```

```
self.__filling(int(h / 2), int(w / 2))
self.grid[int(h / 2)][int(w / 2)] = 2
self.__filling_outside()

@property
def cells(self):
    r = []
    for i, row in enumerate(self.grid):
        for j, cell in enumerate(row):
            r.append([j, i, cell])
    return r

@property
def outside(self):
    return [cell for cell in self.cells if cell[2] == 1]

@property
def d_outside(self):
    return np.array([[x * (0.02 / w), y * (0.015 / h), v] for x, y, v in self.cells if v == 1])

@property
def inside(self):
    return np.array([cell for cell in self.cells if cell[2] == 2])

@property
def d_inside(self):
    return np.array([[x * (0.02 / w), y * (0.015 / h), v] for x, y, v in self.cells if v == 2])

@property
def walls(self):
```

```
        return np.array([cell for cell in self.cells if cell[2] == 3])

@property
def d_walls(self):
    return np.array([[x * (0.02 / w), y * (0.015 / h), v] for x, y, v in self.cells if v == 3])

def is_outline(self, cell):
    return self.grid[cell[1]][cell[0]] == 1

def is_inside(self, cell):
    return self.grid[cell[1]][cell[0]] == 2

def is_true_outside(self, cell):
    return self.grid[cell[1]][cell[0]] == 3

def get_coords(self, i, j):
    return i * self.s_w, j * self.s_h

def __filling(self, i, j):
    neigs = [(i + 1, j), (i - 1, j), (i, j - 1), (i, j + 1)]
    for pixel in neigs:
        if 0 <= pixel[0] < self.h and 0 <= pixel[1] < self.w:
            if self.grid[pixel[0]][pixel[1]] == 0:
                self.grid[pixel[0]][pixel[1]] = 2
                self.__filling(pixel[0], pixel[1])

def __filling_outside(self):
    for (i, j, _) in self.outside:
        neigs = [(i + 1, j), (i - 1, j), (i, j - 1), (i, j + 1)]
        for pixel in neigs:
```

```
        if 0 <= pixel[1] < self.h and 0 <= pixel[0] < self.w and self.is_inside(pixel):
            self.grid[j][i] = 3
            break

def plot(self, show_outline=False):
    for cell in self.cells:
        if self.is_outline(cell) and show_outline:
            plt.gca().add_patch(
                plt.Rectangle(self.get_coords(cell[0], cell[1]), self.s_w, self.s_h, fill=True,
                               color='b'))
        if self.is_inside(cell):
            plt.gca().add_patch(
                plt.Rectangle(self.get_coords(cell[0], cell[1]), self.s_w, self.s_h, fill=True,
                               color='r'))
        elif self.is_true_outside(cell):
            plt.gca().add_patch(
                plt.Rectangle(self.get_coords(cell[0], cell[1]), self.s_w, self.s_h, fill=True,
                               color='g'))
        else:
            plt.gca().add_patch(
                plt.Rectangle(self.get_coords(cell[0], cell[1]), self.s_w, self.s_h, fill=False,
                               lw=0))
    plt.title("Points du contour")
    plt.axis('equal')
    plt.show()
```

```
import sys
import numpy as np

import json

import torch
import torch.nn as nn
from torch.optim.lr_scheduler import StepLR

from src.data import AneurysmData
from src.utils import increase_points_number, parabolic_inlet, ns_residuals

sys.setrecursionlimit(10000)

w, h = 150, 90
data = AneurysmData("../input/datasets/eliepernet/solid-77-vtk/Solid_77.vtk", w, h)

rho = 1055.0
mu = 0.0035
nu = mu / rho

V_max = 1.114
L_ref = 0.02
P_ref = rho * V_max ** 2 # Pa
Re = V_max * 0.004 / nu
nu = 1 / Re
inlet_radius = 0.09

def to_tensor(xy_np):
    return torch.tensor(np.float64(xy_np), dtype=torch.float64)
```

```
inside_points = list(np.float64(np.column_stack((data.d_inside[:, 0] / L_ref, data.d_inside[:, 1] /
↪ L_ref))))
inside_points = torch.tensor(inside_points).double()

inside_points = increase_points_number(inside_points, 4, 0.001, 0.001)
inside_points = inside_points.detach().requires_grad_(True)

outside_points = torch.tensor(
    np.float64(np.column_stack((data.d_walls[:, 0] / L_ref, data.d_walls[:, 1] / L_ref))))).double()

inlet_points = torch.column_stack((
    torch.linspace(0.1, 0.285, 150),
    torch.zeros(150)
)).double()
inlet_points = inlet_points.detach().requires_grad_(True)

inlet_labels = torch.column_stack((
    torch.zeros(len(inlet_points)),
    parabolic_inlet(inlet_points[:, 0].detach())
)).double().detach()

outlet_mask = (data.inside[:, 1] == 0) & (data.inside[:, 0] > 50)
outlet_xy = data.inside[outlet_mask, :2] / L_ref # ← cohérent
outlet_points = to_tensor(outlet_xy)

outside_points = increase_points_number(outside_points, 8, 0.001, 0.001).detach()
outlet_points = increase_points_number(outlet_points, 16, 0.001, 0.001).detach()

PINNModel = nn.Sequential(
    nn.Linear(2, 128), nn.Tanh(),
```



```

    nn.Linear(128, 128), nn.Tanh(),
    nn.Linear(128, 128), nn.Tanh(),
    nn.Linear(128, 128), nn.Tanh(),
    nn.Linear(128, 3)
).double()

def initialize_weights(module):
    if isinstance(module, nn.Linear):
        nn.init.xavier_uniform_(module.weight, gain=1.0)
        nn.init.zeros_(module.bias)

PINNModel.apply(initialize_weights)

mse = nn.MSELoss()
losses = []

def loss_function():
    inside_values = PINNModel(inside_points)
    ns_x, ns_y, continuity = ns_residuals(inside_points, inside_values[:, 0], inside_values[:, 1],
    ↪ inside_values[:, 2],
                                   nu)

    ns_loss = mse(ns_x, torch.zeros_like(ns_x)) + mse(ns_y, torch.zeros_like(ns_y))
    cont_loss = mse(continuity, torch.zeros_like(continuity))

    wall_values = PINNModel(outside_points)
    wall_loss = mse(wall_values[:, 0], torch.zeros_like(wall_values[:, 0])) \
        + mse(wall_values[:, 1], torch.zeros_like(wall_values[:, 1]))

```

```
# Inlet
inlet_values = PINNModel(inlet_points)
inlet_loss = mse(inlet_values[:, 0], inlet_labels[:, 0]) \
             + mse(inlet_values[:, 1], inlet_labels[:, 1])

outlet_values = PINNModel(outlet_points)
outlet_loss = mse(outlet_values[:, 2], torch.zeros_like(outlet_values[:, 2]))

total = (
    50 * (ns_loss
          + cont_loss)
    + 10 * wall_loss
    + 50 * inlet_loss
    + 5 * outlet_loss
)

losses.append([ns_loss.item(), cont_loss.item(), wall_loss.item(),
               inlet_loss.item(), outlet_loss.item(), total.item()])
return total

lossses = []

adam_epochs = 10000

adam_optimizer = torch.optim.Adam(PINNModel.parameters(), lr=0.001)
scheduler = StepLR(adam_optimizer, step_size=15000, gamma=0.1)

for epoch in range(adam_epochs):
```

```
adam_optimizer.zero_grad()

total_loss = loss_function()
total_loss.backward()

adam_optimizer.step()
scheduler.step()

optimizer_lbfgs = torch.optim.LBFGS(
    PINNModel.parameters(),
    lr=1.0,
    max_iter=1000,
    history_size=50,
    tolerance_grad=1e-7,
    tolerance_change=1e-9,
    line_search_fn="strong_wolfe"
)

def closure():
    optimizer_lbfgs.zero_grad()
    loss = loss_function()
    loss.backward()
    return loss

torch.save(PINNModel.state_dict(), "./AN_model_01.pt")

with open("AN_model_01.json", "w") as f:
    json.dump(losses, f)
```

```
import json

import numpy as np
import torch
import torch.nn as nn

from src.utils import extract_ds_values, ns_residuals

H5_FILE = "../coarse_dataset/AllFields_Resultats_MESH_210.h5"
T = 8 # pas de temps 20

RE = 520
nu = 1 / RE

Q = 7

P = -3700 + 1.32 * Q
P_ref = abs(P)
P_norm = P / P_ref

U_ref = 1000

model = nn.Sequential(
    nn.Linear(2, 256),
    nn.Tanh(),
    nn.Linear(256, 256),
    nn.Tanh(),
    nn.Linear(256, 256),
    nn.Tanh(),
```

```

        nn.Linear(256, 3)
    ).double()

optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

mse = nn.MSELoss()
losses = []
adam_epochs = 10000

x, y, uv = extract_ds_values(H5_FILE, T)

inside_points = torch.tensor(
    np.column_stack((x / 18, y / 14)),
).double().requires_grad_(True)

inside_labels = torch.tensor(uv / U_ref).double()
outlet_points = torch.column_stack((torch.linspace(14.5 / 18, 17.85 / 18, 50),
↪ torch.zeros(50))).double()

def compute_total_loss():
    inside_values = model(inside_points)
    inside_values_loss = mse(inside_values[:, :2], inside_labels)

    ns_x, ns_y, continuity = ns_residuals(inside_points, inside_values[:, 0], inside_values[:, 1],
↪ inside_values[:, 2], nu)

    ns_loss_x_inside, ns_loss_y_inside = mse(ns_x, torch.zeros_like(ns_x)), mse(ns_y,
↪ torch.zeros_like(ns_y))
    continuity_loss_inside = mse(continuity, torch.zeros_like(continuity))

    outlet_values = model(outlet_points)
    outlet_values_loss = mse(outlet_values[:, 2], torch.full_like(outlet_values[:, 2],
↪ P_norm).double())

```

```
losses.append([
    inside_values_loss.item(),
    ns_loss_x_inside.item(),
    ns_loss_y_inside.item(),
    continuity_loss_inside.item(),
    outlet_values_loss.item(),
    ns_loss_x_inside.item() + ns_loss_y_inside.item() + continuity_loss_inside.item() +
    ↪ outlet_values_loss.item() + inside_values_loss.item()
])

return ns_loss_x_inside + ns_loss_y_inside + continuity_loss_inside + 20 * outlet_values_loss +
↪ 100 * inside_values_loss

for _ in range(adam_epochs):
    optimizer.zero_grad()
    loss = compute_total_loss()
    loss.backward()
    optimizer.step()

optimizer_lbfgs = torch.optim.LBFGS(
    model.parameters(), lr=1.0, max_iter=50000,
    history_size=100, line_search_fn="strong_wolfe"
)

def closure():
    optimizer_lbfgs.zero_grad()
    loss = compute_total_loss()
    loss.backward()
    return loss

optimizer_lbfgs.step(closure)
```

```
torch.save(model.state_dict(), "./DS_Model_03.pt")  
  
with open("DS_Model_03.json", "w") as f:  
    json.dump(losses, f)
```

```
import json

import numpy as np
import torch
import torch.nn as nn

from src.utils import extract_ds_values, increase_points_number, parabolic_inlet,
↳ extract_ds_outside_values, \
   ns_residuals

H5_FILE = "../input/datasets/eliepernet/mesh-210/AllFields_Resultats_MESH_210.h5"
MODEL_FILE = "../input/models/eliepernet/modelds/pytorch/default/1/DS_Model_03.pt"

T = 20

V_max = 0.777
P_fact = 0.521

Re = 377
nu = 1 / Re

PINNModel = nn.Sequential(
    nn.Linear(2, 256),
    nn.Tanh(),
    nn.Linear(256, 256),
    nn.Tanh(),
    nn.Linear(256, 256),
    nn.Tanh(),
    nn.Linear(256, 3)
).float()
PINNModel.load_state_dict(torch.load(MODEL_FILE))
```



```
frozen_params = {n: p.clone().detach() for n, p in PINNModel.named_parameters()}

optimizer_transfer = torch.optim.Adam(
    PINNModel.parameters(),
    lr=1e-5
)

mse = nn.MSELoss()
losses = []

x, y, _ = extract_ds_values(H5_FILE, T)

inside_points = torch.tensor(
    np.column_stack((x / 20, y / 14)),
).float()

inside_points = increase_points_number(inside_points, 4, 0.002, 0.002)
inside_points = inside_points.detach().requires_grad_(True)

inlet_points = torch.column_stack((
    torch.linspace(0.1, 0.285, 150),
    torch.zeros(150)
)).float()
inlet_points = inlet_points.detach().requires_grad_(True)

inlet_labels = torch.column_stack((
    torch.zeros(len(inlet_points)),
    parabolic_inlet(inlet_points[:, 0].detach(), V_max)
)).float().detach()
```

```
x_out, y_out = extract_ds_outside_values(H5_FILE)
outside_points = torch.tensor(
    np.column_stack((x_out / 20, y_out / 14))
).float()
outside_points = outside_points[outside_points[:, 1] > 0.001].detach()

outlet_points = torch.column_stack((
    torch.linspace(14.5 / 20, 17.85 / 20, 150),
    torch.zeros(150)
)).float().detach()

def compute_total_loss(epoch, n_epochs, lambda_anchor=5e3):
    inside_values = PINNModel(inside_points)

    # BC
    inlet_values = PINNModel(inlet_points)
    outside_values = PINNModel(outside_points)
    outlet_values = PINNModel(outlet_points)

    inlet_loss = mse(inlet_values[:, :2], inlet_labels)
    outside_loss = mse(outside_values[:, :2], torch.zeros_like(outside_values[:, :2]))
    outlet_loss = mse(outlet_values[:, 2], torch.full_like(outlet_values[:, 2], P_fact))

    bc_loss = 50 * outside_loss + 50 * inlet_loss + 5 * outlet_loss

    ns_x, ns_y, continuity = ns_residuals(
        inside_points,
        inside_values[:, 0], inside_values[:, 1], inside_values[:, 2],
        nu
```

```
)
ns_loss = mse(ns_x, torch.zeros_like(ns_x)) \
          + mse(ns_y, torch.zeros_like(ns_y))
cont_loss = mse(continuity, torch.zeros_like(continuity))

w_pde = min(1.0, max(0.0, (epoch - 0.2 * n_epochs) / (0.4 * n_epochs)))
pde_loss = w_pde * (ns_loss + cont_loss)

pinn_loss = bc_loss + pde_loss

anchor_loss = sum(
    (p - frozen_params[n]).pow(2).sum()
    for n, p in PINNModel.named_parameters()
)

total = pinn_loss + lambda_anchor * anchor_loss

losses.append([
    total.item(),
    ns_loss.item(),
    cont_loss.item(),
    inlet_loss.item(),
    outside_loss.item(),
    outlet_loss.item(),
    anchor_loss.item(),
    w_pde,
    lambda_anchor
])

return total, w_pde
```

```
def get_lambda(epoch, n_epochs, lambda_start=1e5, lambda_end=1e2):
    t = epoch / n_epochs
    return lambda_start * (lambda_end / lambda_start) ** t

transfer_epochs = 8000

for epoch in range(transfer_epochs):
    optimizer_transfer.zero_grad()
    lam = get_lambda(epoch, transfer_epochs)
    loss, w_pde = compute_total_loss(epoch, transfer_epochs, lambda_anchor=lam)
    loss.backward()
    optimizer_transfer.step()

last_losses = [l[0] for l in losses[-200:]]

variation = max(last_losses) - min(last_losses)
if variation < 0.05 * np.mean(last_losses):
    print("Adam stable → lancement L-BFGS")

    for p in PINNModel.parameters():
        p.requires_grad = True

    optimizer_lbfgs = torch.optim.LBFGS(
        PINNModel.parameters(), lr=1.0,
        max_iter=500, history_size=50,
        line_search_fn="strong_wolfe"
    )
```

```
def closure_transfer():
    optimizer_lbfgs.zero_grad()
    loss, _ = compute_total_loss(transfer_epochs, transfer_epochs, lambda_anchor=0.0)
    loss.backward()
    return loss

for step in range(20):
    optimizer_lbfgs.step(closure_transfer)

torch.save(PINNModel.state_dict(), "./DS_Model_03.pt")

with open("DS_Model_03.json", "w") as f:
    json.dump(losses, f)
```